Conestoga College – SET

Project Report

Application Security

Name: **Eric Moutoux, Will Jessel, Zemmat Hagos**

Date of Submission: 2026-3-22

SEF

(SECU - 2000)

Table of Contents

Summary of Work 3
- Tasks / Milestones 3
  - User Design 3
  - System design 3
  - Basic System design 4
- Code samples / details 4
- Accomplishments 4
  - System design 4

Work In Progress 5
- Current Tasks 5
  - Which parts of the system is being done. 5
  - Who's doing what 5
  - Timeline 5
  - Start date 5
- Current Testing sections 5
  - Which parts of the system is being tested. 5
  - Who's doing it 6

Next Steps 6
- Upcoming Features 6

Issues, Risks, or Challenges 6
- Roadblocks 6
  - Programmatic Delays 6
  - Non-Programmatic Delays 7
- Dependencies 7
  - External libraries 7
  - Programming languages 7
  - Database server and queries 7

Summary of Work

Tasks / Milestones

User Design

Creating a user

Creating a user should include a username, password and country of residence. Firstly, the username is to help identify the user and give them a sense of personal expression within the system. Then the password is to keep the user protected within the system by forcing users to create strong and enforce password character requirements such as:

Password:

Minimum 10 characters long

At least 5 nonalphanumeric characters

No white space

EX: as:A>#R”:Aasdk

Not only this hashing the password before it is stored onto the database will allow for secure and harder to hack or accounts to become compromised. Finally, the country of residence will be used to determine what prices the user should be subjected to. Whether its American or Canadian prices as this will be what is supported in our program at the start.

Logging in as a user

Logging in to a user account will behalf much like other web-based stores in which you are presented a menu to input a username and password. Should a user forget their password they are screwed as we are not implementing the email password reset system in this current iteration.

System design

Core security principles

1. Confidentiality **(maybe)**

Multi-factor authentication: will protect users accounts by sending a code to a email or phone after a successful login to stop hackers who have found out the password from entering the account.

Sessions: will be locked to a user’s device stopping hackers from hijacking their session for nefarious actions.

2. Integrity

Input scrubbing: Using regex and validation checks to remove white space and illegal characters such as ; or <>. This way no query syntax characters enter the database.

Sanitizing Output: before showing the user any feedback to the screen cleans it before displaying to the user while also keeping text feed back vague enough to hide business logic or unintentional user login credentials but also guiding users to using the system

3. Availability

Rate Limiting: Set a limit in which on how fast users can attempt to login as to not flood servers with bad requests. However still allowing users to attempt to login should they fail.

Error Handling: If the app crashes or hits an error, make sure it doesn't show business secrets to the user. But showing generic messages like "server error" to hide what faulted on the back end.

Basic System design

One of the main system design assumptions made while creating this system is assuming all user input boxes or items are vulnerable and unsafe. As such having both client side and deeper server-side validation for the inputs from the boxes to protect the server and database from outside interference.

Another main system design is developing a least privilege system. In which users who create accounts are automatically flagged as users in which only have the bare minimum ability to buy and download games.

Overall, this will create a layered defense that will protect our inner database and system from outside attackers that do not need or are allowed to see of interact with these directly.

Code samples / details

- Add screenshots and explanations of code screenshots

Accomplishments

System design

The current system state is partially finished. The rest Api is set up along side the SQL connection. use of procedures within the SQL side to stop users from seeing how usernames are found by only returning a Boolean to determine if a password and username was matched whilst also logging the login attempt on the account. These procedures will read the login list history to determine if its suspicious by returning a Boolean to let the server know if the login attempts should be blocked. A basic JavaScript/HTML front end that allows login testing and basic validation. This allows us to gather a basic use case of the program:

Creating an account
Login to an account
Buying a game
"Download a game"

A keynote about the download game functionality is that we won't be implementing it as we don't have data saved on a database like fully functional games. As such we are going to create a text file that simply is called the game in which the user downloads to mimic the user downloading the game onto their computer without forcing scope creep into this assignment.

Work In Progress

Current Tasks

Which parts of the system is being done.

The current work is being done on the front end to make it more user friendly and add further user validation and create a secure front end finish the front end. This will allow us to determine what the serve side functionality should do and finalize what the back end will contain. This will also bring up the use of further procedures or tables within the SQL database should they be needed making it so we don't have to return to the SQL multiple times to see if certain tables are needed to be updated or not.

Who's doing what

Currently each group member takes a functionality from the front-end system and begins updating it from the older basic model. For example, working on the login page or create user page would be two items that group members would do. As the login types and creation of accounts are simple, we have left this to a single group member to allow for the other two members to get a head start on the C# server Api and the game purchase or library pages. Once the first group member is done with the login pages, they will then assist the other two members in creating and testing the future tasks that need to be finished on the system

Login/create account pages: Zemmat Hagos

Front End Library pages: Will Jessel

Back-end Api and SQL database: Eric Moutoux

Timeline

The front-end login accounts pages should take no more than a day or two as such should be done within the end of the weekend. As the other two tasks will take a slightly longer amount of time which will also include future testing and code review this will likely take the rest of the following week. However, once the tasks are complete testing tasks will take up the remaining time until the assignment is due to create a safe and protected system that can protect users and systems alike.

Start date

The start date of the assignment was when the design and implementation was sketched out and building the basic layout following took around a week. Including basic tests. The Current set of creations began in the tail end of march and will finish around April 8th.

Current Testing sections

Which parts of the system is being tested.

The current system being tested is the basic system as each change is made no matter how small. Once each change is made a single tester or multiple testers depending on the test will run the code and attempt to investigate it to see if it introduces security vulnerabilities. This testing includes the SQL

tables and procedures. Account creation and login pages. As further tests on the main pages and Api items will take place once the SQL tables and the account pages are complete.

Who's doing it

Testing is a group effort and is not solely laid upon anyone as having multiple viewpoints on each error or security vulnerability that appears allows multiple solutions and a better solution to be created rather than a solution that simply fixes a bug. As knowing what a hacker or user would be impossible having each member give their input and the programmer who created the bug implement the fix allows for learning as a time and sharing of thought processes that could improve everyone's abilities as the system develops.

Next Steps

Upcoming Features

Next features we are planning to include into this program or future for this program outside of this assignments scope would be adding a forget password to the login screen that emails you a reset password link.

Adding more visual images to the game library. Like adding game art images and other such items. This could also include reviews, game summaries and other user benefiting features. Finally, we could add more options for interacting with games like the company's who created them and other such details that things like steam have.

All future steps within the scope of the project can be found within the above sections including updating the user interface, improving the SQL database structure and defense. Improving data validation and adding further details and Ui abilities to the store and library pages.

Issues, Risks, or Challenges

Roadblocks

Programmatic Delays

Relearning how to create rest Api's once again after being away from them for so long lead to some growing pains as we reimbursed ourselves in connecting a Api to an SQL server. The syntax for [HTTP] requests specifically gave us a small challenge as they aren't as easily picked up on compared to the rest of the c# coding language.

Small syntax mistakes also where challenging as JavaScript and [HTTP] requests tends to be a bit harder to visually check out while looking at it. Connecting to the SQL database is rather quite easy now with our give advanced SQL course allowing this to come up easy however using it within a rest Api

needed us to follow a tutorial to explain how exactly to use a SQL database in the right manner within our systems design.

There were a few times where foreign keys and table item creations were a tad bit buggy but overall worked quiet well as such not many run time errors were encountered

There have yet to be any major logical flow errors and design errors within our program as such have yet to truly hold us back from completing the first objective. However, I Forsee the next steps involving more of those logical flow errors and internal programmatic errors.

Non-Programmatic Delays

While the Ui is very basic now the underlying flow and feel of the system is somewhat what we are hoping for. While this may change as we expand the library section and store front currently as the program stands the logical flow and its non programmatic issues aren't a front facing issue

Dependencies

External libraries

One of the main external libraries are apart of the hashing process as must other functionality are supported natively within C# and JavaScript libraries.

Programming languages

- C# as the server side of our rest Api.
- JavaScript for the front end of the website that the user interacts with
- SQL for setting up and querying the server for user logins or creation of the tables

Database server and queries

The main database server will be called Mist and the tables within this server will be small but effective lowering complexity whilst also keeping security. The first table will be a user or accounts table in which passwords, emails, usernames, ids are stored alongside the user privilege state. The second table will include what games the specific accounts or owns, and the third table will include a table of all available games within the program. That is used to populate the store. The final table is a list of logs that are associated with accounts to determine if multiple login attempts have been made within a short period of time or to determine other security issues for db. users.